

CS305 SDN Project Report

Tong Wu (12412460)

Zhikai Wang (12410615)

Yunzang Li (12410922)

June 4, 2026

1 Project Overview

This project implements a centralized Software-Defined Networking controller on top of os-ken and Mininet. The controller is designed for a single-subnet Layer-2 network and supports three major functions:

- dynamic IP allocation through DHCP,
- shortest-path forwarding with topology awareness,
- packet filtering through switch-level firewall rules.

The main design goal is to keep the control logic inside the controller while leaving packet forwarding to OpenFlow switches created by Mininet. This separation makes it possible to react to host discovery, topology updates, and policy changes in a programmable and testable way.

2 System Architecture

The implementation is organized around the following core files:

- `controller.py`: entry point of the SDN controller. It receives switch, host, link, and packet events, maintains the global network view, installs forwarding rules, and coordinates the DHCP and firewall modules.
- `dhcp.py`: DHCP server implementation, including address pool management, OFFER/ACK packet construction, timeout recovery, lease expiration, and duplicate allocation prevention.
- `firewall.py`: firewall rule loader and rule installation logic. It converts high-level JSON rules into high-priority OpenFlow drop entries.
- `ofctl_utilis.py`: helper utilities for building and sending OpenFlow messages and packets.
- `tests/`: Mininet-based validation scripts for DHCP, switching, firewall, DNS, and bonus experiments.

At runtime, Mininet builds the emulated network, while os-ken runs the controller. Switches connect to the controller through OpenFlow 1.0. Hosts generate DHCP, ARP, ICMP, TCP, and DNS traffic, and the controller decides how the network should react.

3 Implementation Details

3.1 DHCP Module

The DHCP server is implemented as a stateful component. It keeps track of:

- pending offers,
- confirmed leases,
- lease expiration timestamps,
- permanently rejected addresses.

When a DHCP DISCOVER packet arrives, the server selects an available address from the configured pool and creates a pending reservation. When a DHCP REQUEST packet is received, the reservation is confirmed and the controller replies with a DHCP ACK. The implementation also includes practical improvements beyond the minimum baseline:

- configurable address pool, netmask, lease time, and offer timeout through environment variables,
- recovery of expired offers,
- lease expiration and renewal support,
- sequential and conflict-free allocation under concurrent requests.

3.2 Topology Learning and Shortest-Path Switching

The controller maintains a global topology view using switch, host, link, and port events. Hosts are learned from gratuitous ARP packets and ARP requests, while switch-to-switch links are learned from LLDP and topology APIs provided by os-ken.

Once the topology is known, the controller computes shortest paths between switches using a breadth-first search over the current adjacency structure. For each destination host, the controller installs destination-MAC-based forwarding rules on every relevant switch. When the topology changes, the controller clears outdated forwarding rules and installs new ones according to the refreshed shortest paths.

This design achieves:

- shortest-path forwarding in topologies with loops,
- automatic path recovery after link or port changes,
- controller-side ARP handling without broadcast flooding.

3.3 Firewall Module

The firewall is implemented as a policy layer above normal forwarding. Rules are loaded from `firewall_rule.json` and translated into high-priority OpenFlow drop flows. Each rule may match source IP, destination IP, protocol, source port, and destination port.

Because firewall entries are installed with higher priority than the regular destination-MAC forwarding rules, blocked traffic is discarded at the switch before it can follow the normal shortest-path forwarding pipeline. This keeps the policy enforcement simple and efficient.

3.4 Testing and Demonstration Support

The project includes both baseline test scripts and extended demonstration scripts. In addition to the original DHCP, switching, and firewall tests, the project now includes:

- custom DHCP configuration tests,
- pool exhaustion tests,
- lease timeout and renewal tests,
- a large complex topology for shortest-path demonstrations,
- a matching complex topology for firewall demonstrations,
- bonus experiments for DNS, TCP congestion, and bufferbloat.

An integrated script, `run_all_tests.sh`, is used to automate the main regression flow and generate logs for the full project.

4 Team Division of Work

We followed a module-oriented division strategy: each member took long-term ownership of a major subsystem, while the team merged changes continuously to reduce final integration risk.

Member	Main Responsibilities	Key Files
Tong Wu (12412460) Role A: DHCP & Bonus DHCP	DHCP basic workflow, lease management, duplicate allocation prevention, RFC-style option handling, and DHCP-related test cases	<code>dhcp.py</code> , <code>tests/dhcp_test/</code> , DHCP bonus test scripts
Zhikai Wang (12410615) Role B: Topology & Switching	Topology event handling, ARP processing, shortest-path computation, forwarding rule installation, dynamic link update handling, and routing-related bonus extensions	<code>controller.py</code> , <code>ofctl_utilis.py</code> , <code>tests/switching_test/</code> , complex topology scripts
Yunzang Li (12410922) Role C: Firewall & Report/Experiment	Firewall rule loading and installation, experiment-oriented bonus tasks, report organization, screenshots, and demonstration scripts	<code>firewall.py</code> , <code>firewall_rule.json</code> , <code>tests/firewall_test/</code> , <code>report/</code> and experiment scripts

5 Module Interfaces and Collaboration

The three modules were designed to collaborate through explicit interfaces:

- The DHCP module exposes `DHCPServer.handle_dhcp(datapath, port, pkt)`. The controller only needs to detect DHCP packets and delegate processing.
- The switching module maintains the shared controller state: `datapaths`, `ofctls`, `hosts`, `links`, and `link_ports`.

- The firewall module exposes `Firewall.install_rules(ofctls)` and relies on the controller’s switch abstractions to push rules consistently across the network.
- Firewall drop rules are installed with higher priority than the normal forwarding rules so that policy enforcement overrides path forwarding when necessary.

6 Conclusion

The final project delivers a functional SDN controller that combines address allocation, shortest-path forwarding, and policy enforcement in a single os-ken application. The implementation is supported by reproducible Mininet test scripts and a complete report structure for demonstration. The modular division of work also made the project easier to develop, debug, and present as a team.